MOHAN BABU UNIVERSITY

Anveshana Club

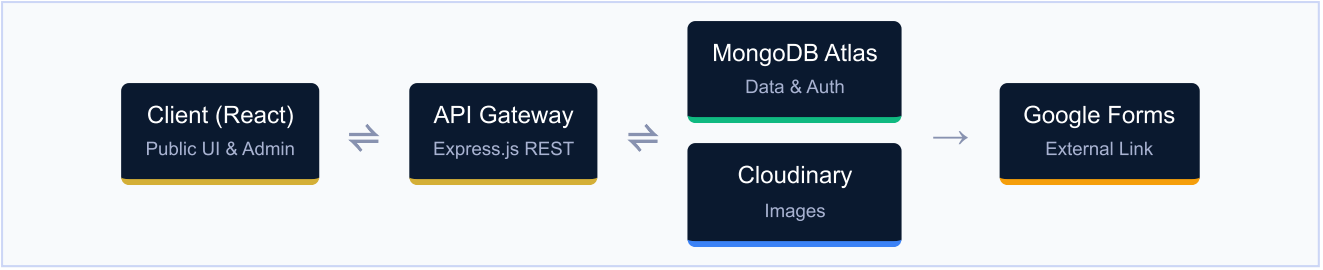
Technical Requirements Document (TRD)

STATUS: APPROVED BASELINE

ALIGNS WITH PRD VERSION 1.0

1. Architecture Overview

The Anveshana Club website utilizes a decoupled MERN architecture to separate presentation from business logic. The public frontend and the secure administrative dashboard are served from a single React application. State is managed via a RESTful Node.js API, which persists data to MongoDB Atlas and offloads binary media to Cloudinary. Event registrations bypass internal systems entirely, linking directly to external Google Forms.



2. Technology Stack

Component	Selection	Reason for Selection	Alternatives & Notes
Frontend	React + Vite (TypeScript)	TypeScript enforces strict contracts between API and UI. Vite provides rapid HMR.	Alt: Next.js (Overkill since SSR is not strictly required for this scale). Free on Vercel.
Styling	Tailwind CSS	Utility-first styling ensures rapid UI development and strict adherence to the Navy/Gold palette without bloat.	Alt: SCSS/MUI. Tailwind prevents CSS scoping collisions in student projects.
Backend	Node.js + Express (TypeScript)	Standard, widely understood by student cohorts. TS ensures schema safety.	Alt: NestJS (Too rigid for MVP). Free tier available on Render/Railway.
Database	MongoDB Atlas + Mongoose	Flexible document model is ideal for evolving event/team structures. ODM adds validation.	Alt: PostgreSQL. Mongo M0 cluster provides 512MB completely free.
Auth	JWT in HttpOnly Cookies	Stateless, scales easily, and HttpOnly cookies mitigate dangerous XSS token theft.	Alt: Redis Sessions. Avoids Redis cost constraint on free tiers.
Media	Cloudinary	On-the-fly optimization and transformation. Offloads bandwidth from the backend.	Alt: AWS S3 (Requires credit card and complex IAM). Cloudinary free tier is generous.

3. Frontend Requirements

3.1 Routing & Architecture

React Router will manage SPA navigation. Public routes include `/`, `/events`, `/events/:id`, `/team`, and `/contact`. The administrative namespace is prefixed with `/admin`. Unauthenticated access to `/admin/*` routes (except the login page) must hard-redirect to the login screen without flashing protected layouts.

3.2 State Management & API Communication

Avoid global Redux. Use **React Query (TanStack Query)** for server state (caching event lists, team profiles, loading states). Use React Context solely for global UI state (e.g., Theme, Admin Auth Status). API calls will be made via Axios, configured to automatically attach credentials (cookies) to outbound requests.

3.3 Validation & Media

- **Form Validation:** Use `Zod` paired with `React Hook Form` for deterministic, type-safe validation on both the contact form and all admin CRUD modals.
- **Image Optimization:** All `<img>` tags must include `loading="lazy"`. Request Cloudinary assets using automatic format and quality parameters (`f_auto,q_auto`).
- **Empty & Error States:** Every data-dependent view must implement a skeleton loader, an empty state (e.g., "No upcoming events"), and a localized error boundary.

4. Backend Requirements

4.1 Application Structure & Conventions

The Express application will follow a strict Controller-Service-Route pattern to decouple business logic from HTTP transport. API endpoints must be prefixed with `/api/v1/` and return standardized JSON responses:

```
{
  "success": true,
  "data": { ... },
  "error": null,
  "meta": { "page": 1, "total": 12 } // For paginated routes
}
```

4.2 Middleware & Error Handling

- **Validation Middleware:** Parse incoming request bodies against Zod schemas before hitting controllers. Invalid payloads immediately return `400 Bad Request` with field-level error arrays.
- **Centralized Error Handling:** A global error middleware catches thrown exceptions, preventing Express from crashing or leaking stack traces to the client in production.
- **Pagination:** `GET /api/v1/events` and `GET /api/v1/feedbacks` must accept `?page=1&limit=10` query parameters. Server responds with bounded subsets.

5. Database Requirements

MongoDB Atlas will house the data. Mongoose schemas must enforce timestamps (`createdAt` , `updatedAt`) implicitly. The following logical collections will be established:

- **Events:** Core entity containing title, description, banner URL, venue, timing, and the external `registrationLink` . Uses a string `status` enum (Upcoming, Open, Closed, Completed).
- **EventGalleries:** To avoid unbound document growth, galleries are a separate collection referencing the `EventId` , holding arrays of Cloudinary image URLs.
- **TeamMembers:** Contains profile data. The `category` field uses a strict enum matching the PRD (Leads, Infra, Design, Data, Social, PR, Tech). A numeric `sortOrder` dictates UI rendering priority.
- **Feedbacks:** Stores 1-5 star ratings and text. Must include IP-hash or timestamp for rate-limiting calculations.
- **Admins:** Stores administrative credentials (email and password hash).

Soft deletion is not required for MVP; administrative deletes will hard-delete records to preserve free-tier storage, but Cloudinary asset cleanup MUST trigger simultaneously via Mongoose pre-remove hooks.

6. Authentication & Authorization

6.1 Admin Provisioning

There is absolutely NO public registration for administrators. The initial admin account will be provisioned via a backend seeder script or environment-variable bootstrap on application start. Subsequent admins can only be created by an existing authenticated admin.

6.2 Secure Session Management

Authentication utilizes JSON Web Tokens (JWT). The process is as follows:

1. Admin submits credentials. Server validates against Argon2 or bcrypt password hashes.
2. Server signs a short-lived JWT (e.g., 2 hours).
3. The token is returned to the client **exclusively inside an HttpOnly, Secure, SameSite=Strict cookie**. It is never exposed in the JSON response body.
4. Frontend determines auth state via a separate `/api/v1/auth/me` ping.

6.3 Authorization Checks

Hiding the frontend `/admin` route is insufficient. Every state-mutating API endpoint (POST, PUT, DELETE) and any route fetching sensitive data (Feedbacks) MUST pass through an `isAuth` Express middleware. This middleware verifies the cookie signature against the server secret before passing execution to the controller.

7. Media Management

MongoDB will not store image binaries. We integrate with Cloudinary using `multer` (to parse multipart/form-data) and a `cloudinary-storage` stream.

- **Upload Restrictions:** File inputs must be limited to `image/jpeg`, `image/png`, and `image/webp`. The Express server must enforce a strict 2MB file size limit per file.
- **Transformations:** Upload requests should instruct Cloudinary to scale massive images down to a maximum width of 1920px prior to storage, preserving the free tier bandwidth.
- **Failure Handling & Deletion:** If a MongoDB document fails to save after an image uploads, the system must trigger a Cloudinary destroy API call to prevent orphaned files. Similarly, deleting an Event must delete its associated Cloudinary folder/assets.

8. API Integration (Google Forms)

As mandated by the PRD, the system relies on Google Forms for event registration to bypass complex PII compliance. The backend will treat the Google Form URL strictly as a string payload within the Event document. The React frontend will render this string in an `<a target="_blank" rel="noopener noreferrer">` tag. The website will NOT attempt to fetch, parse, or display Google Forms responses.

9. Security and Privacy

- **Environment Secrets:** Database URIs, JWT secrets, and Cloudinary keys must never be committed. Use `.env` locally and hosting provider secret managers in production.
- **Input Sanitization:** Use `express-mongo-sanitize` to prevent NoSQL injection via request bodies/queries. Apply `xss-clean` to strip malicious tags from rich-text event descriptions or feedback submissions.
- **Spam Prevention:** The public `POST /api/v1/feedbacks` route must be governed by `express-rate-limit` (e.g., max 3 requests per 15 minutes per IP address) to prevent database flooding.
- **Headers:** Implement `Helmet.js` to set strict Content-Security-Policy (CSP), disable the `X-Powered-By` header, and enforce frameguard protections.
- **CORS:** Configure CORS to accept cross-origin requests ONLY from the exact production frontend domain and localhost during development. Reject all others.

10. Folder Structure

The project will utilize a dual-root monorepo approach to maintain a clear separation of concerns while living in one Git repository.

```
anveshana-web/
├── client/                 # React + Vite Frontend
│   ├── src/
│   │   ├── assets/         # Static icons, global CSS
│   │   ├── components/     # Reusable UI (Buttons, Cards, Nav)
│   │   ├── context/        # Global React Contexts
│   │   ├── pages/          # Route-level components (Home, Admin)
│   │   ├── services/       # Axios API client functions
│   │   └── utils/          # Formatting, constants
│   ├── index.html
│   └── package.json
└── server/                 # Express Backend
    ├── src/
    │   ├── config/         # DB connection, Cloudinary setup
    │   ├── controllers/    # Request/Response logic
    │   ├── middlewares/    # Auth, Validation, Error Handling
    │   ├── models/         # Mongoose Schemas
    │   ├── routes/         # Express Router definitions
    │   └── utils/          # JWT signers, custom error classes
    ├── server.ts           # Entry point
    └── package.json
```

11. Environment Variables

Variable Name	Purpose	Required	Service
`MONGO_URI`	Connection string for MongoDB Atlas cluster.	Yes	Backend
`JWT_SECRET`	Cryptographic key for signing admin session tokens.	Yes	Backend
`CLOUDINARY_URL`	API connection string for media uploads.	Yes	Backend
`CLIENT_URL`	Frontend domain (for strict CORS policy).	Yes	Backend
`VITE_API_URL`	Base URL for Axios requests to the backend.	Yes	Frontend

12. Deployment and Operations

The application will be deployed across robust free-tier/low-cost PaaS providers to accommodate a university club budget.

- **Frontend Hosting:** Vercel. Connected to the `main` branch of the GitHub repo. Automatically triggers `npm run build` on push.
- **Backend Hosting:** Render (Web Service). Automatically builds and starts the Node server. *Note: Free tier web services spin down after inactivity, causing a ~30 second cold start on the first API request.*
- **Database & Media:** Managed entirely by MongoDB Atlas and Cloudinary SaaS tiers.
- **HTTPS/Domain:** Vercel provisions TLS certificates automatically. Custom domain (e.g., `anveshana.mbu.edu.in` or `.com`) will be mapped via CNAME records to Vercel.

13. Testing Requirements

- **Frontend:** Unit test critical helper functions (date formatting, status logic) via Vitest. Ensure the responsive grid breaks cleanly at 768px. Run automated Lighthouse audits to ensure accessibility scores exceed 90.
- **Backend:** Integration test the API using Jest and Supertest. Focus testing effort on the `isAuth` middleware and the Zod validation layers to ensure protected routes cannot be bypassed or fed invalid schemas.

14. Technical Decisions and Risks (ADR)

- **Decision: No Server-Side Rendering (SSR).** *Reasoning:* Anveshana's primary traffic is internal university students discovering the site via direct links or campus posters. Deep SEO is unnecessary. React SPA saves backend compute costs compared to a Next.js Node server.
- **Decision: No Email Server (SMTP).** *Reasoning:* Admin password resets are excluded from MVP to avoid setting up Nodemailer/SendGrid. If an admin loses their password, a database manager must manually overwrite the hash.
- **Risk: Render Cold Starts.** *Mitigation:* The frontend UI must implement a clean loading skeleton on the initial fetch, preventing users from assuming the site is broken during the 30-second backend boot time.
- **Risk: Cloudinary Storage Limits.** *Mitigation:* Event Galleries can consume space quickly. The backend MUST enforce strict image compression and size limits before proxying files to the Cloudinary API.